Documentation Duality and AI Collaboration I

Every directory at every level of the repository hierarchy contains two documentation files:

- `README.md`: Human-readable overview, quick-start instructions, and directory structure.
- `AGENTS.md`: Machine-readable technical specification optimized for AI coding assistants. Contains API tables, dependency graphs, implementation patterns, and architectural constraints.

Documentation Duality and AI Collaboration II

This Documentation Duality standard serves two purposes. First, it ensures that both human researchers and AI agents can navigate the codebase efficiently—`AGENTS.md` files provide the structured context that language models need to make informed code modifications without hallucinating API signatures or violating architectural invariants. Second, it creates a self-documenting feedback loop: as AI agents modify the codebase, they update the corresponding `AGENTS.md` files, keeping documentation synchronized with implementation. Lau and Guo's survey of 90 AI coding assistant systems [@lau2025aicoding] identifies contextual code understanding as a primary bottleneck; the Documentation Duality standard addresses this by providing pre-structured context at every directory level.

Documentation Duality and AI Collaboration III

The template additionally includes `CLAUDE.md` at the repository root, providing system-level instructions for AI coding assistants—architectural principles, testing requirements, and naming conventions that apply globally. This creates a three-tier documentation architecture: per-directory `AGENTS.md` for local context, root `README.md` and `CLAUDE.md` for global constraints, and `README.md` for human navigation.

Agentic Skill Architecture I

The Documentation Duality standard addresses human and AI navigation at the directory level. A complementary layer operates at the *module* level: every infrastructure subpackage carries two additional machine-readable files that transform it from a passive code library into an active, protocol-aligned skill endpoint.

Agentic Skill Architecture II

The Three-Tier Skill Protocol

`template/` implements a progression of agent-facing documentation, escalating in specificity from global constraints to module-level API contracts:

Tier	File	Scope	Purpose
1 — System	`README.md`	Repository root	Global architectural principles, Zero-Mock policy, naming conventions
2 — Structure	`AGENTS.md`	Every directory	Local file inventories, API surfaces, integration patterns, architectural constraints
3 — Skill	`SKILL.md`	Every infrastructure module	Machine-parseable skill descriptor: module name, description, key imports, usage pattern

Tier 1 and Tier 2 have direct analogues in the prompt-engineering literature: system prompts and retrieval-augmented context [@lau2025aicoding]. Tier 3 is novel. The `SKILL.md` files follow a YAML frontmatter + Markdown instruction format precisely aligned with the tool-descriptor schemas of the Model Context Protocol [@anthropic2024mcp]. The following is the exact frontmatter from `infrastructure/rendering/SKILL.md`: